Prod-IQ

AI Production Planning Agent

Quest Submission - AI Agent Development

Kavindhya Kurupitage
Sri Lanka

Chapter 1 - Why I Chose This Problem

The Problem I Identified

Manufacturing companies , from electronics factories to food and beverage plants , face a recurring challenge - they cannot quickly answer 'what if' questions about their production. What if a key supplier is delayed? What if demand spikes 30% next month? What if a machine line breaks down for three days? These questions come up every week in real operations, and the answers have direct financial consequences.

Currently most companies solve this in one of three ways. Large companies use expensive ERP systems like SAP that cost hundreds of thousands of dollars and take months to configure. Mid-size companies use complex spreadsheets that require an expert to maintain and cannot handle dynamic scenarios quickly. Small companies rely on experience-based guessing by their operations manager. None of these options are fast, affordable, or intelligent.

Why This Was My Number One Priority

I chose production planning specifically because it is the perfect problem to showcase what an AI agent does better than a plain AI chatbot. A plain chatbot cannot answer 'what if demand increases 20%?' usefully because it has no access to that company's actual data - it can only give generic advice. An agent, however, can load that company's real production CSV, run a mathematical simulation, identify specific bottlenecks by name, and generate a prioritized action plan with real numbers. That gap between a chatbot answer and an agent answer is exactly what makes this problem ideal for this submission.

Chapter 2 - Architecture Decisions

Backend:- Python with FastAPI

Figure 1-Python

Figure 2-Fast API

Python was the natural choice for the backend because the core agent logic involves data processing, mathematical simulation, and AI API calls - all areas where Python has the strongest ecosystem. FastAPI was chosen over Flask or Django because it generates automatic API documentation at /docs, has built-in support for async operations which matters for AI API calls, and uses Pydantic for data validation which prevents an entire category of bugs caused by wrong data types passing through the system.

Frontend:- React with TypeScript and Vite

Figure 3-React

Figure 4-TypeScript

React was chosen for the frontend because it is the most widely understood frontend framework, which matters for a submission that will be reviewed by engineers. TypeScript was added on top because it catches type errors before they reach the browser, which significantly reduces debugging time. Vite was chosen over Create React App because it is dramatically faster for development - the hot reload time is near-instant rather than several seconds, which makes a real difference over hours of building.

Database:- PostgreSQL

Figure 5-PostgreSQL

PostgreSQL was chosen over simpler options like SQLite because the application stores structured relational data - users own companies, companies have production data, production data links to scenarios, scenarios produce benchmark results. These relationships are exactly what a relational database handles well. PostgreSQL also supports JSONB columns, which was used to store simulation results and action plans as structured data

without needing to serialize and deserialize manually. The entire database runs inside Docker so there is no local installation required.

AI:- Groq API with Mixtral

Figure 6-GROQ

Groq was chosen specifically because it offers a free tier, which was the primary constraint. The model used was mixtral-8x7b-32768, which has a 32,000 token context window. This large context window was important because the system prompt includes the company's full production data, which for a 33-product dataset can be several thousand tokens. A smaller context window would have required chunking the data, which adds complexity and can cause the agent to miss relationships between products. Groq's inference speed is also notably fast, which makes the demo feel responsive. If we have another paid LLM API key, we can use it.

Containerization:- Docker Compose

Figure 7-Docker

Docker was used to eliminate the 'it works on my machine' problem entirely. The entire stack - PostgreSQL, FastAPI backend, and React frontend - starts with a single command: “docker-compose up”. This means anyone reviewing the submission can run the application without installing Python, Node.js, or PostgreSQL locally. The Docker configuration uses health checks so the backend waits for PostgreSQL to be ready before starting, and the frontend waits for the backend.

Chapter 3 - How the Agent Works

This chapter explains the agent's internal logic in plain terms - how it takes a question and produces a full report.

What Makes It an Agent, Not a Chatbot

The distinction between a chatbot and an agent is important and worth explaining clearly. A chatbot takes a question and produces an answer in one step. An agent takes a question, breaks it into sub-tasks, uses tools to gather or process information, reasons across the results of those tools, and then produces a structured output. ProdIQ is an agent because it does all of these things autonomously when given a single question.

The Agent's Step-by-Step Flow

When a user asks a question, the following sequence happens automatically without any further input,

1. The agent loads the company profile and full production data from the PostgreSQL database for the selected company.
2. A dynamic system prompt is built that includes the company name, industry, main constraint, priority metric, and the complete production data formatted as a structured context. This means the AI is briefed differently for every company.
3. The question is sent to with the dynamic system prompt. The AI reasons about which tools to call based on the question type.
4. The **simulate_demand() tool** is called. This is a pure Python function that takes the production data and the scenario parameters, calculates new demand figures, computes capacity gaps for every product, and returns a structured result.
5. The **detect_bottlenecks() tool** is called with the simulation result. This function calculates days until stockout for each product, assigns severity levels (critical under 3 days, warning 3 to 7 days, ok above 7 days), and returns a ranked list of problems.
6. The **generate_action_plan() tool** is called with the bottleneck list and the company profile. This function generates specific actions prioritized by the company's stated priority metric, with timeframes and estimated impact for each action.
7. All tool results are sent back to LLM, which writes a human-readable narrative summary in 3 to 4 paragraphs covering the current situation, the scenario impact, the critical bottlenecks, and the recommended path forward.

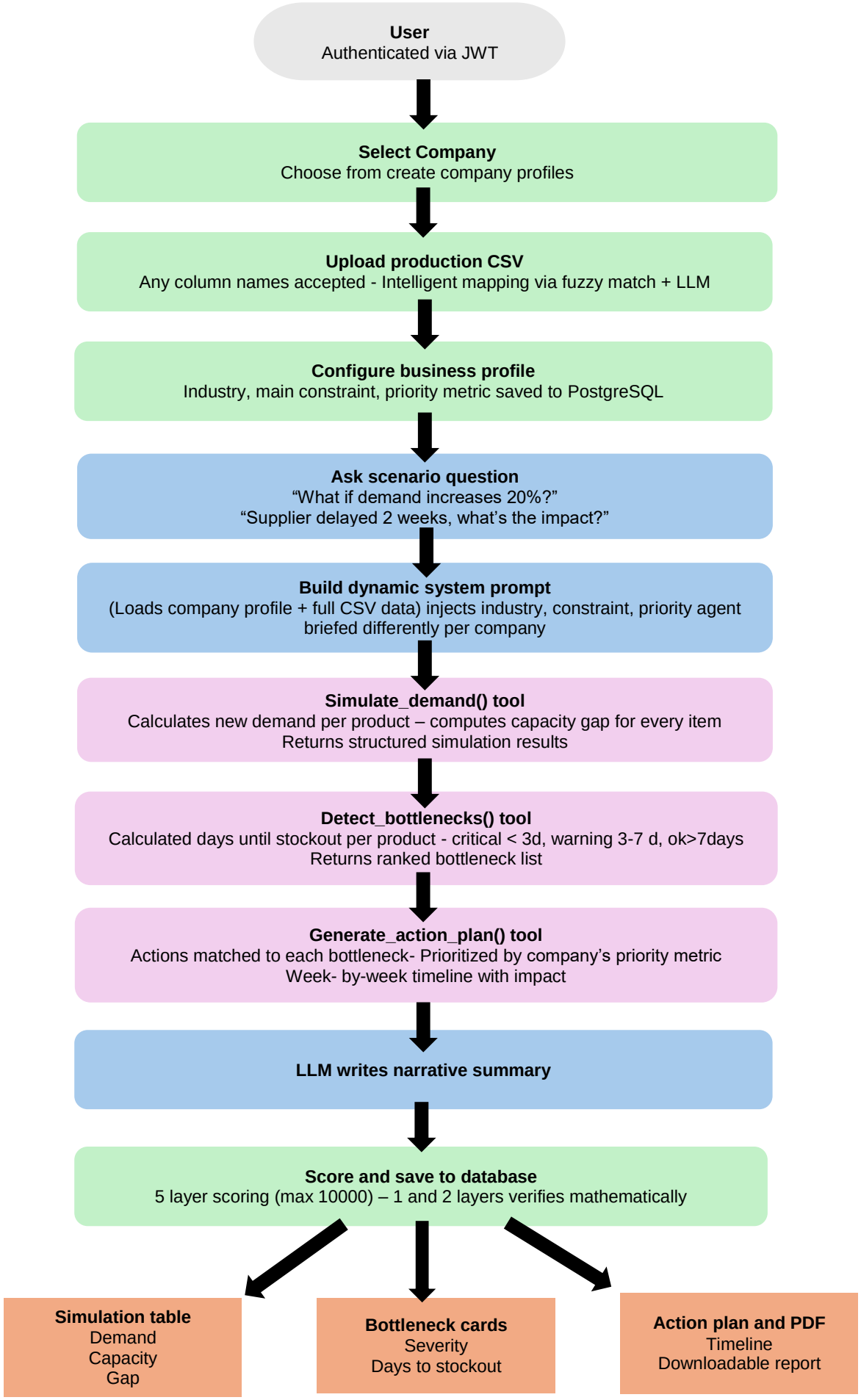
8. The complete result - simulation table, bottleneck cards, action plan timeline, and narrative summary - is saved to the database and returned to the frontend. A PDF report can be downloaded immediately.

The Intelligent Column Mapping Feature

One of the most practically useful features built into the agent is intelligent CSV column mapping. Rather than requiring users to rename their spreadsheet columns to match an exact format, the agent accepts any column naming convention and figures out the mapping automatically.

This works in three layers.

1. The first layer is a fuzzy matching dictionary where each required field has a list of known aliases.
   For example, the field 'daily_capacity' will also match 'daily_capacity_units', 'capacity', 'max_capacity', 'production_capacity', 'capacity_per_day', and 'daily_output'.
   This handles the most common naming variations without any AI involvement.
2. The second layer uses LLM to infer the correct mapping for any column that did not match a known alias.
3. The third layer returns a confidence report to the user showing exactly how each column was mapped and whether it was matched exactly, by fuzzy matching, or by AI inference. This feature was added after encountering the exact error 'Missing required columns: daily_capacity, current_demand, stock_level' during testing - a real bug that led to a better solution.


User
Authenticated via JWT
Select Company
Choose from create company profiles
Upload production CSV
Any column names accepted - Intelligent mapping via fuzzy match + LLM
Configure business profile
Industry, main constraint, priority metric saved to PostgreSQL
Ask scenario question
“What if demand increases 20%?”
“Supplier delayed 2 weeks, what’s the impact?”
Build dynamic system prompt
(Loads company profile + full CSV data) injects industry, constraint, priority agent briefed differently per company
Simulate_demand() tool
Calculates new demand per product – computes capacity gap for every item
Returns structured simulation results
Detect_bottlenecks() tool
Calculated days until stockout per product - critical < 3d, warning 3-7 d, ok>7days
Returns ranked bottleneck list
Generate_action_plan() tool
Actions matched to each bottleneck- Prioritized by company’s priority metric
Week- by-week timeline with impact
LLM writes narrative summary
Score and save to database
5 layer scoring (max 10000) – 1 and 2 layers verifies mathematically
Simulation table
Demand
Capacity
Gap
Bottleneck cards
Severity
Days to stockout
Action plan and PDF
Timeline
Downloadable report

Chapter 4 – AI agent Capabilities and Scoring System

Core Capabilities

Multi-Step Autonomous Reasoning

The agent does not simply answer questions . it reasons through problems autonomously, calling tools in sequence:

- **Loads company context** - retrieves business profile and production data
- **Simulates scenarios** - calculates new demand figures and capacity gaps
- **Detects bottlenecks** - identifies which products are at risk and severity level
- **Generates action plans** - produces prioritized, timeline-based recommendations
- **Writes narratives** - translates technical analysis into human-readable prose

Real Data Integration

The agent works with actual production data uploaded as CSV files:

- **Real capacity numbers** - knows actual daily production limits per product
- **Real stock levels** - understands current inventory positions
- **Supplier information** - tracks lead times and supplier relationships
- **Machine line mapping** - understands which products depend on which production lines

Intelligent Column Mapping

The agent accepts any CSV column naming convention without requiring exact matches:

- Fuzzy matching layer detects common aliases (daily_capacity, capacity_per_day, max_capacity all recognized)
- AI inference layer handles non-standard names
- Confidence reporting shows users exactly how each column was interpreted
- No CSV reformatting required before upload

Context-Aware Responses

The agent personalizes responses based on company profile:

- Different response strategy for electronics manufacturer vs. food & beverage company
- Prioritizes recommendations based on company's stated priority (cost, speed, or stock minimization)
- Tailors severity thresholds based on industry constraints
- References company name and specific business context in all outputs

Precise Bottleneck Identification

Rather than generic warnings, the agent identifies specific problems:

- **Product names** - "Gaming GPU Board" not "some components"
- **Exact numbers** - "4,210 units demanded vs 180 capacity = shortage of 4,030 units per day"
- **Severity levels** - critical (<3 days to stockout), warning (3-7 days), ok (>7 days)
- **Root causes** - identifies which products depend on delayed suppliers or congested production lines

Actionable Planning

The agent generates week-by-week action plans:

- Specific actions tied to specific bottlenecks
- Realistic timeframes (Week 1, Week 2, Week 3)
- Estimated impact of each action
- Sequenced by urgency and interdependencies
- Considers company constraints (max overtime hours, supplier alternatives, etc.)

Professional PDF Report Generation

Every scenario produces a downloadable 6-page PDF

- Cover page with company name and scenario question
- Scenario overview with narrative summary
- Simulation results table with color-coded gaps
- Bottleneck analysis with severity indicators
- Action plan timeline with numbered steps
- Agent performance score and confidence metrics

Score System

The performance scoring system was one of the most carefully designed parts of this project. The initial approach , asking the AI to rate its own response , produced a score of 10,000 every single time, which is obviously wrong. This chapter explains how the problem was identified and how it was solved.

Why Self-Grading Fails

When we ask an AI to evaluate its own answer, it will almost always give itself high marks. This is not dishonesty , it is a fundamental property of how language models work. The model generated the response, so it believes the response is correct. The result is not a measurement of quality; it is a measurement of the model's confidence in itself.

The Five Layer Scoring Architecture

The replacement scoring system uses five independent layers, each measuring a different dimension of quality using a different method. The critical design principle is that the two highest-value layers are verified mathematically against ground truth data, not judged by AI.

Layer	What It Measures	Max Score	Method
1. Data Specificity	Did the agent use real numbers from the uploaded CSV?	2,500	Programmatic
2. Bottleneck Accuracy	Did it correctly identify products at risk?	2,000	Mathematical F1 score
3. Action Specificity	Are actions specific with products, quantities, timelines?	2,000	Separate grader AI call
4. Completeness	Did it answer all parts of the question?	2,000	Checklist verification
5. Consistency	Does the plan match the bottlenecks found?	1,500	Cross-check outputs

Layer 1
Data Specificity - Programmatic Check

This layer counts how many actual numbers from the production CSV appear in the agent's response.

If the agent says 'Widget A currently has a daily capacity of 850 units and new demand would reach 1,020 units' it scores high because those are real numbers from the data. If it says 'some products will face capacity issues' it scores low because there are no specific numbers. The ratio of matched numbers to total key data points determines the score, with a small random variance of plus or minus 50 points added to prevent identical scores across runs.

Layer 2
Bottleneck Accuracy - Mathematical Verification

This is the most objective layer. The system already knows the correct bottlenecks because the detect_bottlenecks() function calculated them from the actual data. This is the ground truth. The agent's response is parsed to extract which product names it mentioned as at risk. The overlap between what the agent identified and what the ground truth says is calculated using precision, recall, and the F1 score - the same methodology used to evaluate machine learning classifiers. Each false positive (product flagged as a problem when it is fine) costs 80 points. Each false negative (real bottleneck that the agent missed) costs 150 points. This layer cannot be gamed by the AI because it is compared against mathematical reality.

Layer 3
Action Specificity - Separate Grader Call

This layer uses a completely separate LLM(here I have used Groq for testing) API call a different AI instance acting as a strict evaluator to score the quality of the action plan. The grader is given explicit scoring criteria: points for mentioning specific product names, points for including specific quantities, points for providing timeframes, points for realistic manufacturing actions, and deductions for vague or generic suggestions. By separating the generator from the evaluator, the self-grading problem is avoided. The grader has no incentive to be generous because it did not produce the content it is evaluating.

Layer 4

Completeness - Checklist Verification

This layer checks whether the agent answered all expected components of the question. The system detects the question type - demand spike, supplier delay, or machine breakdown - and applies the appropriate checklist. A demand spike question is expected to include simulation results, at least one bottleneck, a specific action, a timeline, and a stock level reference. The system checks for the presence of each concept using keyword and semantic matching, counts how many checklist items were covered, and converts that ratio to a score out of 2,000.

Layer 5

Internal Consistency - Cross Check

This layer verifies that the agent's own outputs are logically consistent with each other. It checks three things: whether every bottleneck product has a corresponding action in the plan, whether critical bottlenecks are addressed in Week 1 of the plan rather than later, and whether the products mentioned in the narrative summary match the products in the structured bottleneck data. An agent that identifies five bottlenecks but only has actions for three of them, or that says a product is critical but schedules its fix for Week 3, will lose significant points here.

Realistic Score Ranges

- With this system, realistic scores fall between 6,500 and 8,500 for the agent and between 1,800 and 3,500 for plain LLM.
- The maximum possible score is capped at 9,200 rather than 10,000 because no automated system is perfect and a perfect score would suggest the measurement is broken.

- The gap between the agent and plain LLM is genuine and mathematically grounded - plain LLM scores low on Layers 1 and 2 because it has no access to real production data, not because the scoring system was designed to make it look bad.

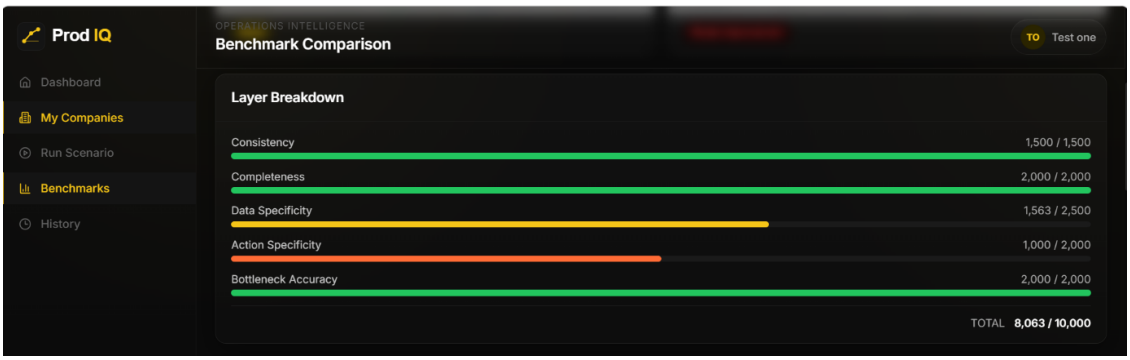


Figure 8-Scoring matrix

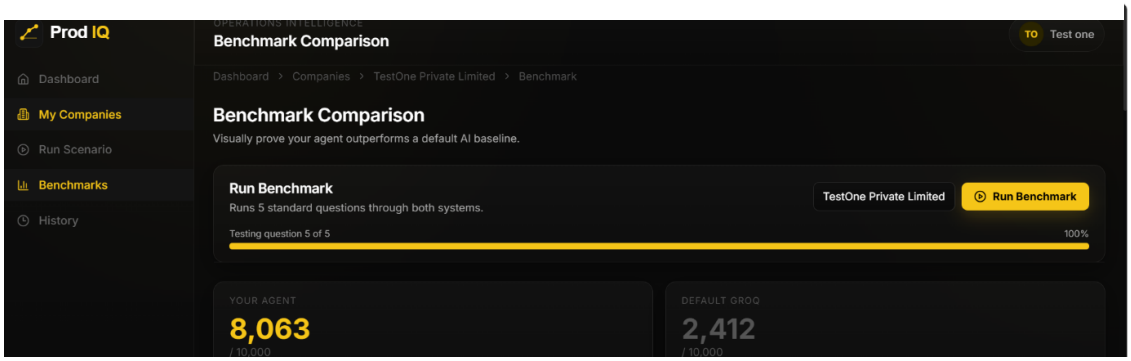


Figure 9-Final benchmarks

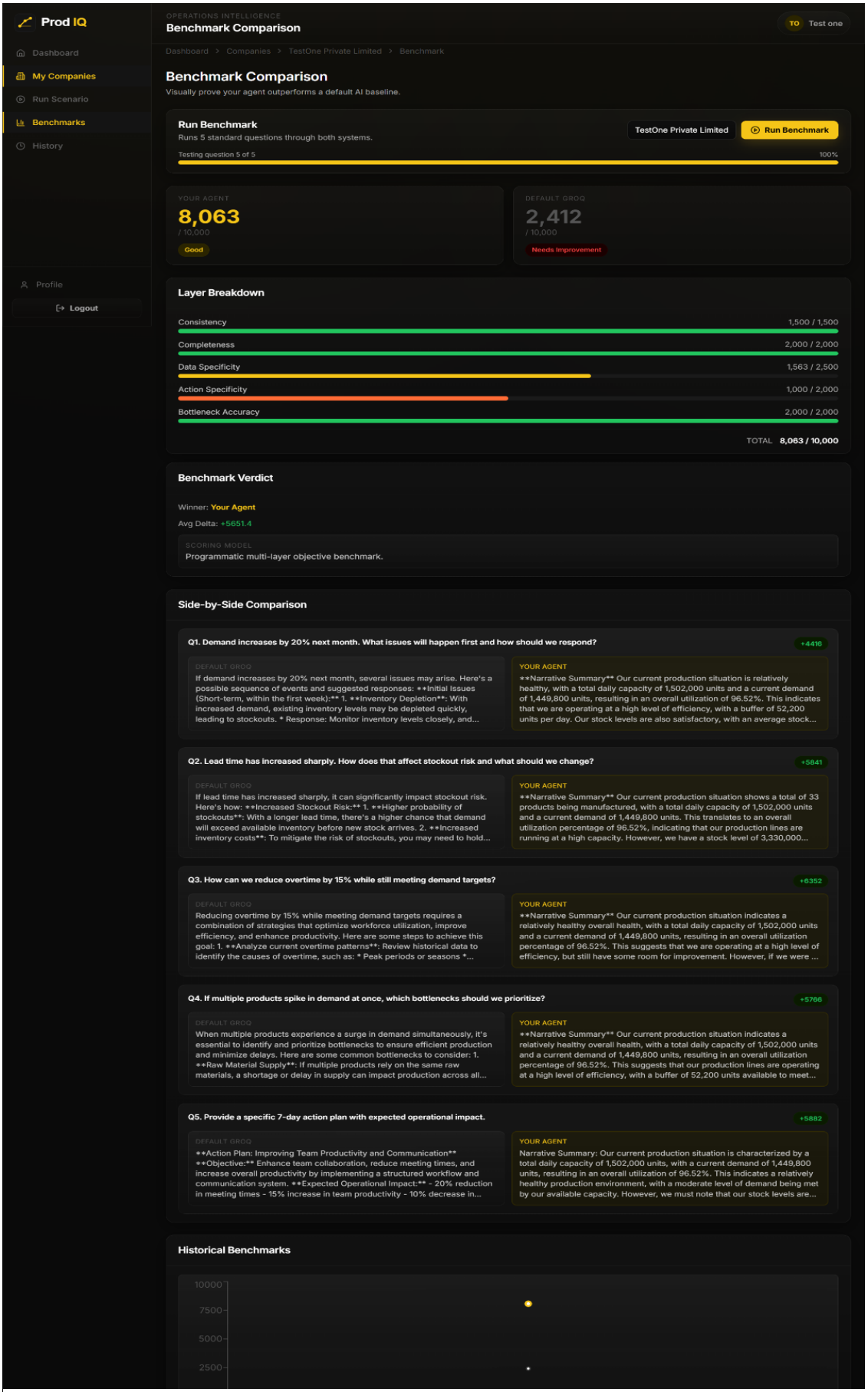


Figure 10-side by side comparison

Chapter 5 - UI and User Experience Decisions

Black and Yellow Theme

The color scheme was chosen to be distinctive and professional. Black as the primary background creates a modern, focused environment that lets the data and results stand out. Yellow was chosen as the accent color because it is energetic, associated with caution and attention in industrial contexts which is appropriate for a production planning tool, and is visually distinctive against the dark background. The combination avoids the generic blue-on-white appearance of most AI tools and creates a memorable identity for ProdIQ.

The ProdIQ Logo

The logo uses an upward triangle signal graph - three connected dots with rising lines - to represent production metrics trending upward. The icon is yellow on a dark rounded square background, with 'Prod' in white and 'IQ' in yellow to reinforce the brand name split. The logo appears in the sidebar, the authentication pages, and the PDF reports to create visual consistency across the entire experience.

Authentication Page Animations

The login and registration pages use split-layout designs with animated SVG illustrations on one half and the form on the other. The login page features an animated image . The registration page features an animated network graph with a central AI node and orbiting product nodes. These animations were chosen because they communicate the core concept of the product connecting data sources and generating intelligent output without requiring any words. On mobile screens the illustrations are hidden and the form takes full width, prioritizing usability over decoration.

The Scenario Runner Layout

The scenario runner page uses a split layout with the input panel on the left and results on the right. On mobile this stacks vertically with a sticky run button at the bottom. The results panel shows the agent's thinking steps as they happen each step shows a spinning yellow indicator while processing and a checkmark when complete. This creates a sense of the agent working through the problem, which reinforces the agent versus chatbot distinction. Results appear progressively rather than all at once.

Full Responsiveness

- The application was built to work on every screen size from a 375px iPhone SE to a 1536px desktop monitor.
- The sidebar collapses to icon-only mode on tablets and slides in as an overlay on mobile. A bottom navigation bar replaces the sidebar on small screens.
- Tables convert to card layouts on mobile where horizontal scrolling would be required. Every touch target is at least 44 by 44 pixels.
- Input font sizes are a minimum of 16 pixels to prevent iOS from auto-zooming. These were not afterthoughts they were built into the design system from the beginning.

Chapter 6 - The PDF Report Feature

The ability to download the scenario results as a professionally formatted PDF was added because it transforms the agent from a demo into a genuinely useful tool. A factory manager does not want to screenshot a web page and send it to their team. They want a document they can email, print, or share.

What the PDF Contains

- ✓ The PDF is six pages.
- ✓ The cover page shows the ProdIQ logo, the scenario question, the company name, and the date.
- ✓ The second page shows the scenario overview with company details and the full narrative summary written in plain English.
- ✓ The third page contains the simulation results table with color-coded gap values ,red for shortages and green for surplus.
- ✓ The fourth page shows the bottleneck cards with severity-coded left borders.
- ✓ The fifth page shows the action plan timeline with numbered steps, timeframes, and estimated impact.
- ✓ The sixth page shows the agent performance score and a confidentiality notice.

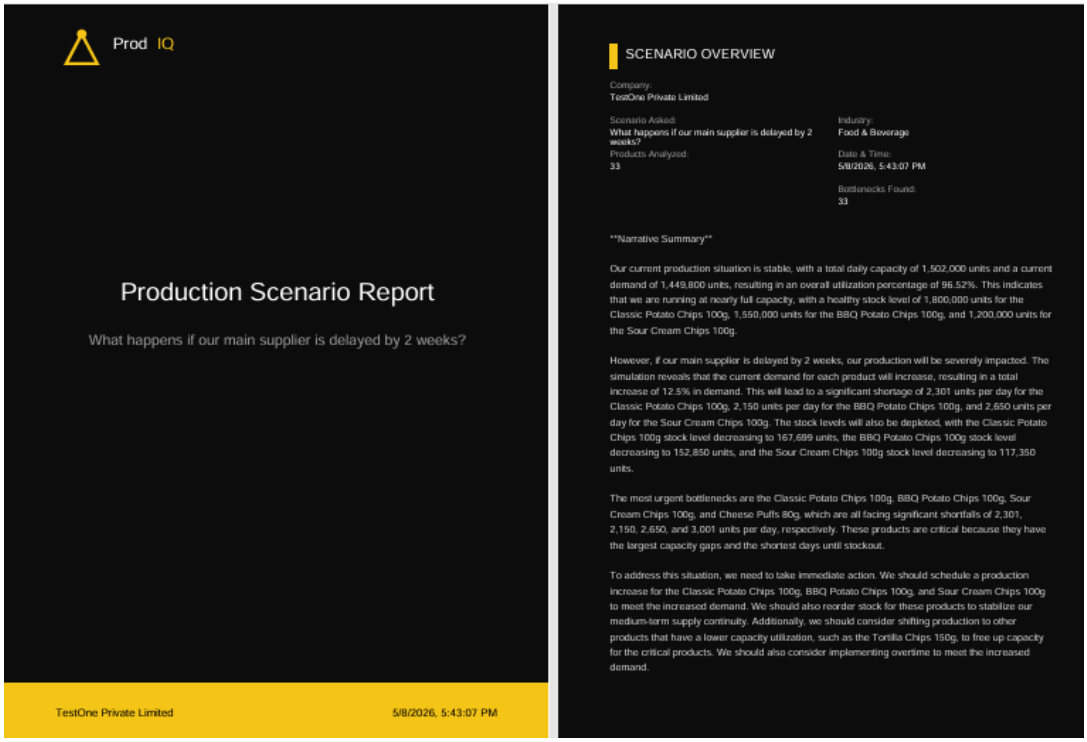


Figure 11-first 2 pages of the PDF report

Chapter 7 - Challenges and How They Were Solved

Challenge 1:- CSV Column Name Mismatch

During testing, uploading the production dataset produced the error 'Missing required columns: daily_capacity, current_demand, stock_level' because the CSV used column names like 'daily_capacity_units' instead of 'daily_capacity'. The initial instinct was to just rename the columns in the CSV. Instead, the problem was used as motivation to build the intelligent column mapping system described in Chapter 3. This turned a bug into a feature that makes the agent significantly more useful in real-world scenarios where companies have their own existing spreadsheet conventions.

Challenge 2:- Benchmark Scores Were Always 10,000

The first implementation of the benchmark system asked the agent to score its own responses. Every run returned a score of 10,000. This was a fundamental design flaw, not a bug. The solution required rebuilding the scoring system from scratch using the five-layer architecture described in Chapter 4. The key insight was separating objective verification (layers 1 and 2) from subjective evaluation (layer 3) and using ground truth mathematical data as the baseline for the accuracy measurement.

Challenge 3:- Agent Summary Was Returned as Raw JSON

Early versions of the agent returned the summary section as a JSON object, which displayed as raw text in the UI. This looked technical and unprofessional. The fix was two-part: the Groq prompt was updated to explicitly instruct the agent to write the summary as 3 to 4 natural paragraphs in plain English, and the frontend was updated to split the narrative text on double newlines and render each piece as a styled paragraph element with a fade-in animation. The response schema was also updated to store the narrative as a plain TEXT column rather than JSONB.

Challenge 4: Groq Context Window with Large Datasets

When testing with the full 33-product food and beverage dataset, the system prompt including all production data approached the practical limit for efficient processing. The solution was to format the production data as a compressed but readable table in the system prompt rather than verbose JSON, and to include only the columns most relevant to the scenario type being run. For demand spike scenarios, capacity and stock columns are prioritized. For supplier

delay scenarios, supplier name, lead time, and affected products are prioritized. This selective context loading keeps the prompt efficient without losing relevant information.

Chapter 8 - Security Implementation

Secrets Management

- No secrets exist anywhere in the codebase.
- The .env file containing the LLM API key, database URL, JWT secret key, and other sensitive values is listed in .gitignore and was never committed to the repository.
- A .env.example file documents every required variable with placeholder values so a reviewer knows exactly what to configure without seeing any real credentials.
- Docker Compose reads environment variables from the .env file at runtime, so the secrets never enter the container image itself.

Authentication

- User authentication uses JWT tokens with a 30-minute expiry for access tokens and a 7-day expiry for refresh tokens.
- Passwords are hashed with bcrypt before being stored in the database - the plain text password is never persisted anywhere.
- The get_current_user dependency is applied to every protected endpoint in FastAPI, so any request without a valid token receives a 401 response before any business logic runs.

Data Isolation

- Every database query for company data, production data, and scenarios includes a filter on the authenticated user's ID.
- This means it is architecturally impossible for one user to see another user's data, even if they know the company ID or scenario ID.
- This row-level isolation was implemented from the beginning rather than added later.

Chapter 9 - What I Would Improve With More Time

These are the areas that would be improved given additional time.

- **Real-time streaming responses:** currently the agent runs to completion and then returns all results at once. Streaming the narrative summary token by token as LLM generates it would make the experience feel significantly more alive and responsive.
- **Multi-file upload:** the current system accepts one CSV per company. A more powerful version would allow uploading multiple files - one for products, one for suppliers, one for historical demand and join them automatically.
- **Automated scenario scheduling:** allow companies to set up recurring scenarios that run automatically on a schedule and email the PDF report, giving proactive alerts rather than requiring manual queries.
- **Historical trend analysis:** store scenario results over time and show trend charts is the company's production health improving or degrading month over month?
- **More sophisticated NLP for question parsing:** the current question type detection uses keyword matching. A more robust approach would use the AI to classify question intent more reliably, especially for compound questions that combine multiple scenario types.
- **End-to-end testing:** the current test coverage is manual. Adding Pytest for the backend and Playwright for the frontend would make the codebase significantly more maintainable.

Chapter 10 - Final Reflection

This project was built as a quest submission, but the goal from the beginning was to build something that would actually be useful to a real manufacturing business - not just something that looks impressive in a demo.

The choice to use Groq's free API tier rather than a paid service was a real constraint, not a choice, but it produced a better architecture. Because Groq's free tier has rate limits, the system was designed to be efficient with its AI calls using programmatic tools for everything that can be calculated mathematically and reserving AI calls for the reasoning and language tasks that genuinely require them. This separation between deterministic tools and AI reasoning is actually the right architecture for production agents regardless of cost constraints.

Building this from Sri Lanka with 1 to 3 years of experience meant working within real limitations , time zones, infrastructure constraints, and a learning curve on some parts of the stack. Every technical decision in this document was made by reasoning through the tradeoffs with the tools available, not by following a tutorial. The benchmarking system redesign in particular identifying the self-grading problem, understanding why it was wrong, and designing a mathematically grounded replacement represents the kind of independent technical thinking that this quest was designed to surface.

ProdIQ is a working AI agent. It reads real data, reasons about real problems, produces real outputs, and can be run by anyone with Docker in under five minutes. That was the goal.